

RomeFuzz: Path-aware Directed Greybox Fuzzing via Dyna-Static Indirect Call Analysis

Index Terms—Path diversity, Directed greybox fuzzing, Indirect call, Selective instrumentation, Collaborative fuzzing

I. ROMEFUZZ

A. Dyna-static indirect call analysis

Collaborative fuzzing. We coordinate fuzzers by sharing test cases that increase global branch coverage. As shown in Figure 1, we design an extensible architecture with components deployed in independent containers to ensure high availability.

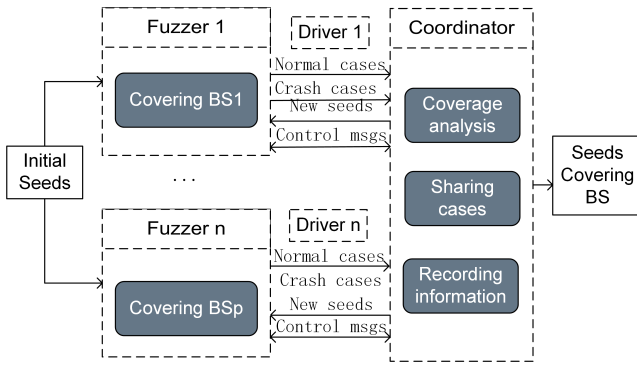


Fig. 1. The overview of collaborative fuzzing. Each dashed box represents an independent component deployed in a Docker container.

Dynamic indirect call identification. We identify function calls following the workflow in Figure 2.

Calling info analysis. We traverse the annotated traces to extract calling relationships via Algorithm 1.

II. EVALUATION

A. Evaluation setup

Benchmarks. As shown in Table II, We selected eight real-world programs to reproduce 21 known bugs including four from MAGMA [1]. Their URLs are provided in Table I.

B. Performance across time Budgets (RQ1)

We evaluated the effectiveness and efficiency of RomeFuzz in detecting known bugs by the following steps. First, we generated a minimized *formal seed* corpus by the refinement procedure in Section III-D. Second, we used these formal seeds as initial inputs for all fuzzers to ensure fairness. Third, we compared RomeFuzz against the four baselines with statistical tests. The results with the time budgets of 3, 6, and 9 hours are shown in Table III, Table IV, and Table V respectively.

Algorithm 1: Identifying function calls.

Input: lines: all lines with calling depths.
Output: mcalls: map between caller functions and the structures composed of a callee function and a call site basic block.
Data: mcs: map between a caller function and a call site basic block.
Data: ecaller: event of caller function.
Data: caller: caller function.
Data: bbcaller: entry basic block of caller function.
Data: dcaller: calling depth of caller function.

```

1 for  $i \leftarrow 0$  to  $\text{len}(\text{lines}) - 1$  do
2    $(\text{ecaller}, \text{caller}, \text{bbcaller}, \text{dcaller}) \leftarrow \text{lines}[i]$ 
3   if  $\text{ecaller} \neq \text{"entry"}$  then
4     continue
5   else
6      $\text{mcalls}[\text{caller}] \leftarrow \text{CREATEMAPCALLS}()$ 
7      $\text{mcs}[\text{caller}] \leftarrow \text{bbcaller}$ 
8   end
9   for  $j \leftarrow i + 1$  to  $\text{len}(\text{lines}) - 1$  do
10     $\text{mcalls}[\text{caller}] \leftarrow$ 
11       $\text{SAVECALLIFNEWCALLEE}()$ 
12       $\text{EXITLOOPWHENCALLEREXIT}()$ 
13       $\text{PROCESSCALLEEEXIT}()$ 
14    if  $\text{INTERNALCALLERBB}()$  then
15      update  $\text{mcs}[\text{caller}]$ 
16  end
17 end

```

TABLE I
THE URLS OF BENCHMARK PROGRAMS.

Benchmark	URL
addr2line	https://sourceware.org/git/binutils-gdb.git
libpng	https://github.com/glennrp/libpng.git
libsndfile	https://github.com/libsndfile/libsndfile.git
libtiff	https://gitlab.com/libtiff/libtiff.git
libxml2	https://gitlab.gnome.org/GNOME/libxml2.git
mjsissues57	https://github.com/cesanta/mjs.git
mjsissues78	https://github.com/cesanta/mjs.git

<pre> hello.c 161 int main(int argc, char *argv[]) { if(a==5){ }else if(a==1 a==2){ 219 printTimeAndStr(a, b,...); 220 void (*functionPtr)(int, ...t); functionPtr = &testswitch; functionPtr(a,b,b); } } </pre>	<pre> hello.c 91 void testswitch(int a, int b, int bb){ printTimeAndStr(a, b, "..."); switch(bb){ case 7: 106 func_after_target_seven(a); break; default: break; } } </pre>	<pre> hello.c void func_after_target_seven(int a){ 50 #ifdef MAGMA_ENABLE_CANARIES MAGMA_TRY 52 { abort(); }MAGMA_CATCH_ALL { #ifdef MAGMA_AVOIDDING exit(0); #else magma_log("BUG002",1); #endif } MAGMA_ENDTRY } </pre>	<pre> src/printzxy.c 5 void printTimeAndStr(...){ 12 snprintf(str,...); 20 printf("%s",str); fputs(str, fp); 24 fclose(fp); } src/c_try_catch.c 33 int frameDef(Frame*) { 35 assert(frame != NULL); 36 return frame->flag; } </pre>
---	--	---	---

Instrumentation & Execution

```

event:entry@@@@@func:main@@@@@bbinfo:entry@@@ln:161--fl:hello.c
...
event:internal@@@@@func:main@@@@@bbinfo:if.else30@@@@ln:219--fl:hello.c
event:internal@@@@@func:main@@@@@bbinfo:lor.lhs.false@@@@ln:219--fl:hello.c
event:internal@@@@@func:main@@@@@bbinfo:if.then33@@@@ln:220--fl:hello.c △
...
event:entry@@@@@func:testswitch@@@@@bbinfo:entry@@@@ln:91--fl:hello.c △
event:entry@@@@@func:printTimeAndStr@@@@@bbinfo:entry@@@@ln:5--fl:src/printzxy.c
event:internal@@@@@func:printTimeAndStr@@@@@bbinfo:if.end@@@@ln:12--fl:src/printzxy.c
event:internal@@@@@func:printTimeAndStr@@@@@bbinfo:if.end11@@@@ln:20--fl:src/printzxy.c
event:exit@@@@@func:printTimeAndStr@@@@@bbinfo:return@@@@ln:24--fl:src/printzxy.c
event:internal@@@@@func:testswitch@@@@@bbinfo:sw.bb2@@@@ln:106--fl:hello.c △
event:entry@@@@@func:func_after_target_seven@@@@@bbinfo:entry@@@@ln:50--fl:hello.c △
event:entry@@@@@func:frameDef@@@@@bbinfo:entry@@@@ln:33--fl:src/c_try_catch.c
event:internal@@@@@func:frameDef@@@@@bbinfo:if.then@@@@ln:35--fl:src/c_try_catch.c
event:exit@@@@@func:frameDef@@@@@bbinfo:if.end@@@@ln:36--fl:src/c_try_catch.c
event:internal@@@@@func:func_after_target_seven@@@@@bbinfo:if.then@@@@ln:52--fl:hello.c

```

Deduplication

```

event:entry@@@@@func:main@@@@@bbinfo:entry@@@@ln:161--fl:hello.c
...
event:internal@@@@@func:main@@@@@bbinfo:if.then33@@@@ln:220--fl:hello.c △
...
event:entry@@@@@func:testswitch@@@@@bbinfo:entry@@@@ln:91--fl:hello.c △
event:entry@@@@@func:printTimeAndStr@@@@@bbinfo:entry@@@@ln:5--fl:src/printzxy.c
event:internal@@@@@func:printTimeAndStr@@@@@bbinfo:if.end11@@@@ln:20--fl:src/printzxy.c
event:exit@@@@@func:printTimeAndStr@@@@@bbinfo:return@@@@ln:24--fl:src/printzxy.c
event:internal@@@@@func:testswitch@@@@@bbinfo:sw.bb2@@@@ln:106--fl:hello.c △
event:entry@@@@@func:func_after_target_seven@@@@@bbinfo:entry@@@@ln:50--fl:hello.c △
event:entry@@@@@func:frameDef@@@@@bbinfo:entry@@@@ln:33--fl:src/c_try_catch.c
event:internal@@@@@func:frameDef@@@@@bbinfo:if.then@@@@ln:35--fl:src/c_try_catch.c
event:exit@@@@@func:frameDef@@@@@bbinfo:if.end@@@@ln:36--fl:src/c_try_catch.c
event:internal@@@@@func:func_after_target_seven@@@@@bbinfo:if.then@@@@ln:52--fl:hello.c

```

Adding depth information

```

event:entry@@@@@func:main@@@@@bbinfo:entry@@@@ln:161--fl:hello.c @@@@@@depth:1
...
event:internal@@@@@func:main@@@@@bbinfo:if.then33@@@@ln:220--fl:hello.c @@@@@@depth:1
...
event:entry@@@@@func:testswitch@@@@@bbinfo:entry@@@@ln:91--fl:hello.c @@@@@@depth:2
event:entry@@@@@func:printTimeAndStr@@@@@bbinfo:entry@@@@ln:5--fl:src/printzxy.c @@@@@@depth:3
event:internal@@@@@func:printTimeAndStr@@@@@bbinfo:if.end11@@@@ln:20--fl:src/printzxy.c @@@@@@depth:3
event:exit@@@@@func:printTimeAndStr@@@@@bbinfo:return@@@@ln:24--fl:src/printzxy.c @@@@@@depth:3
event:internal@@@@@func:testswitch@@@@@bbinfo:sw.bb2@@@@ln:106--fl:hello.c @@@@@@depth:2
event:entry@@@@@func:func_after_target_seven@@@@@bbinfo:entry@@@@ln:50--fl:hello.c @@@@@@depth:3
event:entry@@@@@func:frameDef@@@@@bbinfo:entry@@@@ln:33--fl:src/c_try_catch.c @@@@@@depth:4
event:internal@@@@@func:frameDef@@@@@bbinfo:if.then@@@@ln:35--fl:src/c_try_catch.c @@@@@@depth:4
event:exit@@@@@func:frameDef@@@@@bbinfo:if.end@@@@ln:36--fl:src/c_try_catch.c @@@@@@depth:4
event:internal@@@@@func:func_after_target_seven@@@@@bbinfo:if.then@@@@ln:52--fl:hello.c @@@@@@depth:3

```

Calling information identification

<pre> "caller": "main", "callsitebb": "if.then33@@@@ln:220--fl:hello.c", "calleees": ["printTimeAndStr", "testswitch"] </pre>	<pre> "caller": "testswitch", "callsitebb": "entry@@@@ln:91--fl:hello.c", "calleees": ["printTimeAndStr"] </pre>	<pre> "caller": "testswitch", "callsitebb": "sw.bb2@@@@ln:106--fl:hello.c", "calleees": ["func_after_target_seven"] </pre>	<pre> "caller": "func_after_target_seven", "callsitebb": "entry@@@@ln:50--fl:hello.c", "calleees": ["frameDef"] </pre>
---	--	--	--

Fig. 2. Example of simulation and calling information Identification. The basic blocks related to the calling chains, such as *main* → *testswitch* → *func_after_target_seven* are emphasized in blue, while the call site blocks are marked with △. Meanwhile, the lines of internal blocks without relation to function calls are marked with red strikethroughs.

TABLE II

THE DETAIL OF BENCHMARK PROGRAMS AND TARGET BUGS FOR EVALUATION. **HBO** MEANS HEAP-BUFFER-OVERFLOW, **HUAF** DENOTES HEAP-USE-AFTER-FREE, **INTO** REPRESENTS INTEGER-OVERFLOW, **GBO** MEANS GLOBAL-BUFFER-OVERFLOW, **SO** DENOTES STACK-OVERFLOW, AND **OOM** REPRESENTS OUT OF MEMORY.

No.	Benchmark	Commit id Version	Usage	Bug details (label, type, function, file:line)
1	addr2line	2.26.1	Binary address to code line	BUG001, SEGV , <i>slurp_syntab</i> , addr2line.c:147
2				BUG002, SEGV , <i>slurp_syntab</i> , addr2line.c:177
3	libming	b72cc2f	Flash output library	BUG001, SEGV , <i>getString</i> , decompile.c:366
4				BUG002, HBO , <i>OpCode</i> , decompile.c:950
5				BUG003, SEGV , <i>getName</i> , decompile.c:449
6				BUG004, HUAF , <i>decompileIF</i> , decompile.c:2471
7	libpng	a37d483	PNG image files	BUG005, SEGV , <i>outputSWF_TEXT_RECORD</i> , outputscript.c:1470
8				PNG003, SEGV , <i>png_handle_PLTE</i> , pngutil.c:992
9	libsndfile	86c9f9e	Sound files with sampled audio data	SND001, INTO , <i>paf_read_header</i> , paf.c:206
10				SND006, GBO , <i>flac_buffer_copy</i> , flac.c:182
11				SND007, HBO , <i>sf_flac_meta_callback</i> , flac.c:470
12				SND017, HBO , <i>wavlike_read_fmt_chunk</i> , wavlike.c:345
13	libtiff	c145a6c	TIFF Library	SND020, SO , <i>wavlike_read_fmt_chunk</i> , wavlike.c:367
14				SND024, HBO , <i>sf_flac_meta_callback</i> , flac.c:459
15	libxml2	ec6e3ef	XML parser and toolkit	TIF007, HBO , <i>ChopUpSingleUncompressedStrip</i> , tif_dirread.c:5972
16				TIF014, OOM , <i>ChopUpSingleUncompressedStrip</i> , tif_dirread.c:5986
17	mjsissues57	d6c06a6	Restricted JS engine	XML017, HBO , <i>xmlParseInternalSubset</i> , parse.c:8228
18	mjsissues78	9eae0e6	Restricted JS engine	BUG001, SEGV , <i>embed_string</i> , mjs.c:13732
19				BUG003, ABRT , <i>get_cb_impl_by_signature</i> , mjs.c:10446
20				BUG004, SEGV , <i>getprop_builtin_foreign</i> , mjs.c:9263
21				BUG006, HBO , <i>mbuf_init</i> , mjs.c:4925

Results for 3 hours. RomeFuzz demonstrates superior effectiveness by reproducing all 21 target bugs, while AFLGo, SELECTFUZZ, WINDRANGER, and PDGF detects 15, 14, 16, and 10 bugs respectively. This advantage in effectiveness is supported by three key metrics. First, RomeFuzz achieves higher stability, successfully triggering 20, 19, 20, and 21 target bugs in no less repetitions than other fuzzers. Second, it shows significantly higher adequacy by *Hit count*, triggering 10, 10, 11, and 6 bugs $3.99\times$, $4.64\times$, $3.09\times$, and $115.27\times$ more frequently on average than AFLGo, SELECTFUZZ, WINDRANGER and PDGF as seen from *Hit count*. Third, the metric of $\hat{A}_{12}$ for *hit count* confirms the above superiority for RomeFuzz, with 16, 18, 18, and 19 values exceeding 0.50 when compared against baseline fuzzers. In terms of efficiency, RomeFuzz achieves a average 66% speed up than baselines according to μ TTE (Minutes). Overall, these results further confirm that RomeFuzz outperforms state-of-the-art DGF fuzzers in both effectiveness and efficiency for bug reproduction.

Results for 6 hours. Overall, RomeFuzz performs similarly to the 3-hour results in terms of effectiveness and efficiency.

Results for 9 hours. Overall, RomeFuzz performs similarly to the 3-hour and 6-hour results in terms of effectiveness and efficiency.

C. Bug reproduction ability with a neutral seed corpus (RQ1)

To assess the impact of seed selection, we re-evaluated RomeFuzz using a neutral seed corpus—generated via collaborative fuzzing but minimized using standard AFL edge

coverage rather than RomeFuzz-specific metrics. Results in [Table VI](#) confirm that RomeFuzz retains its superior performance even with these non-customized seeds. Moreover, the findings highlight that fuzzing performance is influenced by seed choice, emphasizing the importance of high-quality, diverse seeds. We advocate for a collaborative fuzzing approach to generate comprehensive seed corpora for all fuzzers, including RomeFuzz and baselines, to enhance overall bug detection effectiveness. Since high-quality collaborative seeds benefit all tested fuzzers, we exclude the time for collaborative fuzzing phase, which aims at seed generation and indirect call analysis, from the Time-to-Exposure (TTE) comparison to ensure a fair assessment of the dynamic fuzzing phase. To balance overhead and coverage gains, the collaborative phase adaptively terminates after a 30-minute coverage plateau. In our experiments, this phase lasted approximately 236, 320, 118, 89, 67, 210, and 209 minutes for addr2line, libming, libpng, libsndfile, libtiff, mjs-issue57, and mjs-issue78, respectively, demonstrating its benchmark-adaptive nature. While this phase averages 3 hours, given that industrial fuzzing campaigns typically run for 24+ hours, this one-time investment is justified by the resulting high-quality seeds and program insights.

Results with a neutral seed corpus. RomeFuzz demonstrates superior effectiveness by reproducing all 8 target bugs, while AFLGo, SELECTFUZZ, WINDRANGER, and PDGF detects 4, 6, 4, and 6 bugs respectively. This advantage in effectiveness is supported by three key metrics. First, RomeFuzz achieves higher stability, successfully triggering 7, 7, 8, and 7 target bugs in no less repetitions than other

TABLE III

RESULTS FOR RQ1 WITH THE DETAIL OF BUG REPRODUCTION BY ROMEFUZZ AND BASELINE FUZZERS WITHIN 3 HOURS. THE METRIC $\hat{A}_{12}$ FOR HIT COUNT IS CALCULATED FOR ROMEFUZZ WITH AFLGo, WINDRANGER, AND SELECTFUZZ AS THE BASELINES. *TO* INDICATES FAILURE TO THE CASE WITHIN TIME BUDGET. *NA* INDICATES ONE FUZZER CANNOT ADAPT TO A BENCHMARK.

Benchmark	Fuzzer combination	Success	Hit count	$\hat{A}_{12}$ for hit count	μ TTE (Minutes)
addr2line BUG001 BUG002	RomeFuzz	2/8	5/20	-/-	56/41
	AFLGo	1/10	1/22	0.56(S)/0.30	27/24
	SELECTFUZZ	10/10	10/11	0.20/0.67(M)	4/90
	WINDRANGER	2/10	20/23	0.51/0.28	85/31
	PDGF	0/0	0/0	0.60(S)/0.90(P)	T.O/T.O
libbing BUG001 BUG002 BUG003 BUG004 BUG005	RomeFuzz	10/10/10/10/10	1151/721/1808/18/175	-/-/-/-/-	19/133/37/7/16
	AFLGo	10/10/10/2/7	1570/544/997/2/24	0.14/0.77(P)/0.94(P)/0.96(P)/0.99(P)	20/28/37/349/80
	SELECTFUZZ	10/10/10/0/0	1338/330/480/0/0	0.37/1.00(P)/1.00(P)/1.00(P)/1.00(P)	36/25/135/T.O/T.O
	WINDRANGER	10/10/9/5/7	718/942/902/41/14	0.75(P)/0.27/0.99(P)/0.82(P)/1.00(P)	4/42/81/322/99
	PDGF	N.A/N.A/N.A/N.A/N.A/N.A/N.A/N.A/N.A/N.A	N.A/N.A/N.A/N.A/N.A/N.A/N.A/N.A/N.A/N.A	N.A/N.A/N.A/N.A/N.A/N.A	N.A/N.A/N.A/N.A/N.A/N.A
libpng PNG003	RomeFuzz	10	212	-	0
	AFLGo	10	113	1.00(P)	1
	SELECTFUZZ	10	63	1.00(P)	1
	WINDRANGER	10	105	1.00(P)	1
	PDGF	10	204	0.52(N)	1
libsndfile SND001 SND006 SND007 SND017 SND020 SND024	RomeFuzz	10/10/10/10/10/10	46/28/137/89/64/139	-/-/-/-/-/-	12/13/18/7/8/18
	AFLGo	0/0/0/5/8/0	0/0/0/10/8/0	1.00(P)/1.00(P)/1.00(P)/1.00(P)/1.00(P)/1.00(P)	T.O/T.O/T.O/11/9/T.O
	SELECTFUZZ	0/0/0/10/10/0	0/0/0/27/18/0	1.00(P)/1.00(P)/1.00(P)/1.00(P)/1.00(P)/1.00(P)	T.O/T.O/T.O/93/80/T.O
	WINDRANGER	0/0/0/7/10/0	0/0/0/34/52/0	1.00(P)/1.00(P)/1.00(P)/0.91(P)/0.58(S)/1.00(P)	T.O/T.O/T.O/10/11/T.O
	PDGF	10/10/10/8/0/10	49/35/77/56/0/136	0.43/0.40/0.91(P)/0.68(M)/1.00(P)/0.51(N)	1/8/3/44/T.O/3
libtiff TIF007 TIF014	RomeFuzz	10/10	1267/75	-/-	1/18
	AFLGo	10/0	440/0	1.00(P)/1.00(P)	4/T.O
	SELECTFUZZ	10/0	137/0	1.00(P)/1.00(P)	4/T.O
	WINDRANGER	10/10	495/10	1.00(P)/0.95(P)	2/45
	PDGF	9/5	95/87	1.00(P)/0.82(P)	68/19
libxml2 XML017	RomeFuzz	10	2016	-	1
	AFLGo	10	4052	0.14	2
	SELECTFUZZ	10	886	0.50(N)	12
	WINDRANGER	10	3585	0.48	2
	PDGF	3	3	0.93(P)	1
mjsissues57 BUG001	RomeFuzz	10	617	-	4
	AFLGo	10	817	0.10	26
	SELECTFUZZ	10	703	0.17	7
	WINDRANGER	10	183	0.90(P)	22
	PDGF	10	776	0.30	1
mjsissues78 BUG003 BUG004 BUG006	RomeFuzz	10/10/10	639/63/145	-	1/30/102
	AFLGo	10/9/0	263/26/0	1.00(P)/0.83(P)/1.00(P)	11/42/T.O
	SELECTFUZZ	10/5/5	38/8/26	1.00(P)/0.94(P)/0.86(P)	6/101/157
	WINDRANGER	10/0/4	247/0/24	1.00(P)/0.99(P)/0.87(P)	6/T.O/44
	PDGF	0/0/0	0/0/0	1.00(P)/1.00(P)/1.00(P)	T.O/T.O/285

fuzzers. Second, it shows significantly higher adequacy by *Hit count*, triggering 3, 4, 4, and 5 bugs 92%, 11.79%, 88%, and 2.20% more frequently on average than AFLGo, SELECTFUZZ, WINDRANGER and PDGF as seen from *Hit count*. Third, the metric of $\hat{A}_{12}$ for *hit count* confirms the above superiority for RomeFuzz, with 7, 5, 7, and 6 values exceeding 0.50 when compared against baseline fuzzers. In terms of efficiency, RomeFuzz achieves a average 66% speed up than baselines according to μ TTE (Minutes). Overall, these results further confirm that RomeFuzz outperforms state-of-the-art DGF fuzzers in both effectiveness and efficiency for bug reproduction.

Computing Systems, vol. 4, no. 3, pp. 1–29, 2020.

REFERENCES

- [1] A. Hazimeh, A. Herrera, and M. Payer, “Magma: A ground-truth fuzzing benchmark,” *Proceedings of the ACM on Measurement and Analysis of*

TABLE IV

RESULTS FOR RQ1 WITH THE DETAIL OF BUG REPRODUCTION BY ROMEFUZZ AND BASELINE FUZZERS WITHIN 6 HOURS. THE METRIC $\hat{A}_{12}$ FOR HIT COUNT IS CALCULATED FOR ROMEFUZZ WITH AFLGo, WINDRANGER, AND SELECTFUZZ AS THE BASELINES. *TO* INDICATES FAILURE TO THE CASE WITHIN TIME BUDGET. *NA* INDICATES ONE FUZZER CANNOT ADAPT TO A BENCHMARK.

Benchmark	Fuzzer combination	Success	Hit count	$\hat{A}_{12}$ for hit count	μ TTE (Minutes)
addr2line BUG001 BUG002	RomeFuzz	6/10	9/42	-/-	205/77
	AFLGo	9/10	14/22	0.38/1.00(P)	269/24
	SELECTFUZZ	10/10	11/11	0.20/0.67(M)	4/90
	WINDRANGER	10/10	28/24	0.08/0.90	221/31
	PDGF	0/0	0/0	0.80(P)/1.00(P)	T.O/T.O
libming BUG001 BUG002 BUG003 BUG004 BUG005	RomeFuzz	10/10/10/10/10	1151/721/1808/18/175	-/-/-/-/-	19/133/37/7/16
	AFLGo	10/10/10/2/7	1570/544/997/2/24	0.14/0.77(P)/0.94(P)/0.96(P)/0.99(P)	20/28/37/349/80
	SELECTFUZZ	10/10/10/0/0	1338/330/480/0/0	0.37/1.00(P)/1.00(P)/1.00(P)/1.00(P)	36/25/135/T.O/T.O
	WINDRANGER	10/10/9/5/7	718/942/902/41/14	0.75(P)/0.27/0.99(P)/0.82(P)/1.00(P)	4/42/81/322/99
	PDGF	N.A/N.A/N.A/N.A/N.A/N.A/N.A/N.A/N.A/N.A	N.A/N.A/N.A/N.A/N.A/N.A/N.A/N.A/N.A/N.A	N.A/N.A/N.A/N.A/N.A/N.A/N.A/N.A/N.A/N.A	N.A/N.A/N.A/N.A/N.A/N.A/N.A/N.A/N.A/N.A
libpng PNG003	RomeFuzz	10	270	-	0
	AFLGo	10	119	1.00(P)	1
	SELECTFUZZ	10	71	1.00(P)	1
	WINDRANGER	10	114	1.00(P)	1
	PDGF	10	254	0.49	1
libsndfile SND001 SND006 SND007 SND017 SND020 SND024	RomeFuzz	10/10/10/10/10/10	46/34/169/97/65/182	-/-/-/-/-/-	12/13/18/7/8/18
	AFLGo	0/0/0/5/10/0	0/0/0/10/10/0	1.00(P)/1.00(P)/1.00(P)/1.00(P)/1.00(P)	T.O/T.O/T.O/11/78/T.O
	SELECTFUZZ	5/0/6/10/10/1	10/0/6/35/20/1	1.00(P)/1.00(P)/1.00(P)/1.00(P)/1.00(P)	304/T.O/245/93/80/254
	WINDRANGER	0/0/0/7/10/0	0/0/0/41/56/0	1.00(P)/1.00(P)/1.00(P)/0.94(P)/0.58(S)/1.00(P)	T.O/T.O/T.O/10/11/T.O
	PDGF	10/10/10/9/0/10	50/40/90/58/0/174	0.43/0.36/0.96(P)/0.77(P)/1.00(P)/0.57(S)	1/8/3/44/T.O/3
libtiff TIF007 TIF014	RomeFuzz	10/10	1784/102	-/-	1/18
	AFLGo	10/0	558/0	1.00(P)/1.00(P)	4/T.O
	SELECTFUZZ	10/1	227/1	1.00(P)/1.00(P)	4/333
	WINDRANGER	10/10	649/13	1.00(P)/1.00(P)	2/45
	PDGF	10/5	178/111	1.00(P)/0.80(P)	68/19
libxml2 XML017	RomeFuzz	10	2093	-	1
	AFLGo	10	4202	0.13	2
	SELECTFUZZ	10	886	0.50(N)	12
	WINDRANGER	10	3726	0.50(N)	2
	PDGF	3	3	0.93(P)	1
mjsissues57 BUG001	RomeFuzz	10	1068	-	4
	AFLGo	10	973	0.59(S)	26
	SELECTFUZZ	10	1196	0.10	7
	WINDRANGER	10	360	0.90(P)	22
	PDGF	10	941	0.44	1
mjsissues78 BUG003 BUG004 BUG006	RomeFuzz	10/10/10	765/90/289	-	1/30/102
	AFLGo	10/9/0	321/28/0	0.99(P)/0.98(P)/0.89(P)	11/42/T.O
	SELECTFUZZ	10/6/7	48/12/50	1.00(P)/0.94(P)/0.86(P)	6/117/157
	WINDRANGER	10/0/4	263/0/26	1.00(P)/1.00(P)/0.96(P)	6/T.O/44
	PDGF	0/0/5	0/0/14	1.00(P)/1.00(P)/0.98(P)	T.O/T.O/285

TABLE V

RESULTS FOR RQ1 WITH THE DETAIL OF BUG REPRODUCTION BY ROMEFUZZ AND BASELINE FUZZERS WITHIN 9 HOURS. THE METRIC $\hat{A}_{12}$ FOR HIT COUNT IS CALCULATED FOR ROMEFUZZ WITH AFLGo, WINDRANGER, AND SELECTFUZZ AS THE BASELINES. *T.O* INDICATES FAILURE TO THE CASE WITHIN TIME BUDGET. *N.A* INDICATES ONE FUZZER CANNOT ADAPT TO A BENCHMARK.

Benchmark	Fuzzer combination	Success	Hit count	$\hat{A}_{12}$ for hit count	μ TTE (Minutes)
addr2line BUG001 BUG002	RomeFuzz	9/10	20/62	-/-	298/77
	AFLGo	10/10	17/22	0.62(S)/0.99(P)	280/24
	SELECTFUZZ	10/10	13/13	0.0.73/1.00	4/90
	WINDRANGER	10/10	28/25	0.29/0.96	221/31
	PDGF	0/0	0/0	0.95(P)/1.00(P)	T.O/T.O
libming BUG001 BUG002 BUG003 BUG004 BUG005	RomeFuzz	10/10/10/10/10	1151/721/1808/18/175	-/-/-/-/-	19/133/37/7/16
	AFLGo	10/10/10/2/7	1570/544/997/2/24	0.14/0.77(P)/0.94(P)/0.96(P)/0.99(P)	20/28/37/349/80
	SELECTFUZZ	10/10/10/0/0	1338/330/480/0/0	0.37/1.00(P)/1.00(P)/1.00(P)/1.00(P)	36/25/135/T.O/T.O
	WINDRANGER	10/10/9/5/7	718/942/902/41/14	0.75(P)/0.27/0.99(P)/0.82(P)/1.00(P)	4/42/81/322/99
	PDGF	N.A/N.A/N.A/N.A/N.A	N.A/N.A/N.A/N.A/N.A	N.A/N.A/N.A/N.A/N.A	N.A/N.A/N.A/N.A/N.A
libpng PNG003	RomeFuzz	10	309	-	0
	AFLGo	10	128	1.00(P)	1
	SELECTFUZZ	10	73	1.00(P)	1
	WINDRANGER	10	128	1.00(P)	1
	PDGF	10	270	0.64	1
libsndfile SND001 SND006 SND007 SND017 SND020 SND024	RomeFuzz	10/10/10/10/10/10	46/41/188/99/68/198	-/-/-/-/-/-	12/13/18/7/8/18
	AFLGo	0/0/0/5/10/0	0/0/0/10/10/0	1.00(P)/1.00(P)/1.00(P)/1.00(P)/1.00(P)	T.O/T.O/T.O/111/78/T.O
	SELECTFUZZ	7/1/6/10/10/2	14/1/6/36/20/2	1.00(P)/1.00(P)/1.00(P)/1.00(P)/1.00(P)	328/516/245/93/80/392
	WINDRANGER	0/0/0/7/10/0	0/0/0/41/56/0	1.00(P)/1.00(P)/1.00(P)/0.94(P)/0.65(M)/1.00(P)	T.O/T.O/T.O/10/11/T.O
	PDGF	10/10/10/9/0/10	50/43/102/59/0/205	0.43/0.43/0.97(P)/0.78(P)/1.00(P)/0.51(N)	1/8/3/44/T.O/3
libtiff TIF007 TIF014	RomeFuzz	10/10	2089/147	-/-	1/18
	AFLGo	10/2	654/3	1.00(P)/1.00(P)	4/483
	SELECTFUZZ	10/3	322/3	1.00(P)/1.00(P)	4/362
	WINDRANGER	10/10	798/16	1.00(P)/1.00(P)	2/45
	PDGF	10/6	237/134	1.00(P)/0.80(P)	68/94
libxml2 XML017	RomeFuzz	10	2164	-	1
	AFLGo	10	4433	0.12	2
	SELECTFUZZ	10	886	0.50(N)	12
	WINDRANGER	10	3897	0.42	2
	PDGF	3	3	0.93(P)	1
mjsissues57 BUG001	RomeFuzz	10	1214	-	4
	AFLGo	10	1066	0.91(P)	26
	SELECTFUZZ	10	1592	0.10	7
	WINDRANGER	10	915	1.00(P)	22
	PDGF	10	1020	0.44	1
mjsissues78 BUG003 BUG004 BUG006	RomeFuzz	10/10/10	892/132/431	-	1/30/102
	AFLGo	10/9/0	327/31/0	1.00(P)/0.93(P)/1.00(P)	13/42/T.O
	SELECTFUZZ	10/7/7	54/12/52	1.00(P)/0.99(P)/0.91(P)	6/153/157
	WINDRANGER	10/0/5	295/0/28	1.00(P)/1.00(P)/0.96(P)	6/T.O/112
	PDGF	0/0/6	0/0/34	1.00(P)/1.00(P)/0.96(P)	T.O/T.O/309

TABLE VI

RESULTS FOR RQ1 WITH THE DETAIL OF BUG REPRODUCTION BY ROMEFUZZ AND BASELINE FUZZERS WITH NEUTRAL SEED CORPUS GENERATED FROM COLLABORATIVE FUZZING BUT MINIMIZED BY STANDARD AFL EDGE COVERAGE. THE METRIC $\hat{A}_{12}$ FOR HIT COUNT IS CALCULATED FOR ROMEFUZZ WITH AFLGo, WINDRANGER, AND SELECTFUZZ AS THE BASELINES. *T.O* INDICATES FAILURE TO THE CASE WITHIN TIME BUDGET. *N.A* INDICATES ONE FUZZER CANNOT ADAPT TO A BENCHMARK.

Benchmark	Fuzzer combination	Success	Hit count	$\hat{A}_{12}$ for hit count	μ TTE (Minutes)
addr2line BUG002	RomeFuzz	10	58	-	145
	AFLGo	10	24	1.00(P)	284
	SELECTFUZZ	9	27	0.96(P)	52
	WINDRANGER	10	30	0.94(P)	172
	PDGF	0	0	1.00(P)	T.O
libming BUG004	RomeFuzz	3	9	-	203
	AFLGo	0	0	0.60(S)	95
	SELECTFUZZ	1	1	0.56(S)	257
	WINDRANGER	1	5	0.59(S)	522
	PDGF	N.A	N.A	N.A	N.A
libpng PNG003	RomeFuzz	10	632	-	0
	AFLGo	10	299	1.00(P)	5
	SELECTFUZZ	10	121	1.00(P)	1
	WINDRANGER	10	251	1.00(P)	1
	PDGF	10	292	1.00(P)	1
libsndfile SND007	RomeFuzz	10	184	-	19
	AFLGo	0	0	1.00(P)	T.O
	SELECTFUZZ	8	8	1.00(P)	357
	WINDRANGER	0	0	1.00(P)	T.O
	PDGF	10	107	0.86(P)	3
libtiff TIF014	RomeFuzz	9	249	-	126
	AFLGo	0	0	0.95(P)	T.O
	SELECTFUZZ	0	0	0.95(P)	T.O
	WINDRANGER	0	0	0.95(P)	T.O
	PDGF	5	50	0.85(P)	178
libxml2 XML017	RomeFuzz	0	0	-	T.O
	AFLGo	2	19	0.40	219
	SELECTFUZZ	0	0	0.50(N)	T.O
	WINDRANGER	0	0	0.50(N)	T.O
	PDGF	1	2	0.45	1
mjsissues57 BUG001	RomeFuzz	10	1894	-	1
	AFLGo	10	1548	0.99(P)	3
	SELECTFUZZ	10	2422	0.00	4
	WINDRANGER	10	1489	0.99(P)	2
	PDGF	10	1654	1.00(P)	1
mjsissues78 BUG004	RomeFuzz	4	12	-	113
	AFLGo	0	0	0.70(M)	T.O
	SELECTFUZZ	7	14	0.39	84
	WINDRANGER	0	0	0.70(M)	T.O
	PDGF	1	2	0.66(M)	604